

A new algorithm for computing the minimum Hausdorff distance between two point sets on a line under translation

Banghe Li^a, Yuefeng Shen^{a,b}, Bo Li^{a,b}

^a Center of Bioinformatics & Key Laboratory of Mathematics Mechanization,
Academy of Mathematics and Systems Science, Chinese Academy of Sciences,
Beijing 100080, China

^b Graduate University of Chinese Academy of Sciences, Beijing 100049, China

Information Processing Letters, Vol106(2008), PP. 52-58

Presenter: Hung-Hsi, Chen

Date: Feb. 3, 2009

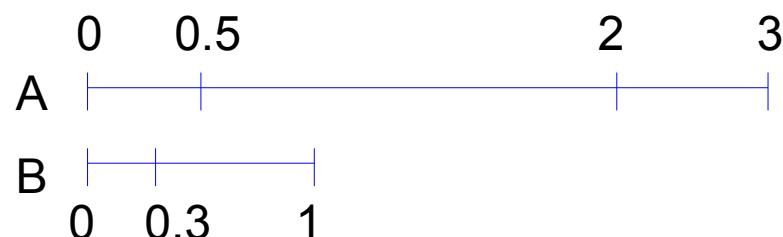
Abstract

To determine the similarity of two point sets is one of the major goals of pattern recognition and computer graphics. One widely studied similarity measure for point sets is the **Hausdorff distance**. So far, various computational methods have been proposed for computing the minimum Hausdorff distance. In this paper, we propose a new algorithm to compute the minimum Hausdorff distance between two point sets on a line under translation, which outperforms other existing algorithms in terms of efficiency despite its complexity of $O((m+n)\lg(m+n))$, where m and n are the sizes of two point sets.

One dimension Hausdorff Distance

$$d(A, B) = \max \left\{ \max_{a \in A} \min_{b \in B} |b - a|, \max_{b \in B} \min_{a \in A} |a - b| \right\}$$

$$A = \{0, 0.5, 2, 3\} \quad B = \{0, 0.3, 1\}$$



$$\begin{aligned} \max_{a \in A} \min_{b \in B} |b - a| &= \max \left\{ \min \{0, 0.3, 1\}, \min \{0.5, 0.2, 0.5\}, \min \{2, 1.7, 1\}, \min \{3, 2.7, 2\} \right\} = \\ &= \max \{0, 0.2, 1, 2\} = 2 \end{aligned}$$

$$\begin{aligned} \max_{b \in B} \min_{a \in A} |a - b| &= \max \left\{ \min \{0, 0.5, 2, 3\}, \min \{0.3, 0.2, 1.7, 2.7\}, \min \{1, 0.5, 1, 2\} \right\} = \\ &= \max \{0, 0.2, 0.5\} = 0.5 \end{aligned}$$

$$d(A, B) = \max \{2, 0.5\} = 2$$

Minimum Hausdorff Distance

$$\min_{t \in \mathbb{R}} d(A + t, B) = \min_{t \in \mathbb{R}} \max \left\{ \max_{a \in A} \min_{b \in B} |b - (a + t)|, \max_{b \in B} \min_{a \in A} |(a + t) - b| \right\}$$

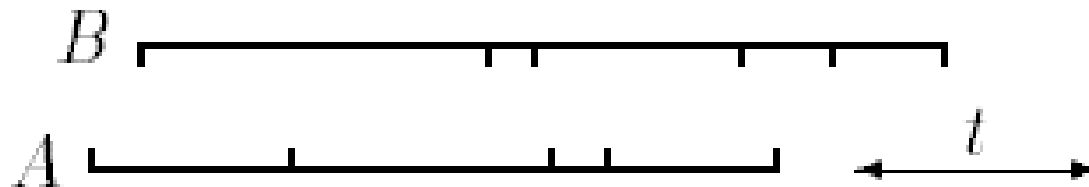


Figure 1: Two sets of points.

Parameter **t** denotes the amount by which A is **shifted**.

Goal: shift set A to get minimum Hausdorff distance

Huttenlocker's Algorithm

Time Complexity: $O(mn \cdot \log(mn))$

$|A|=m, |B|=n$ both are **sorted**

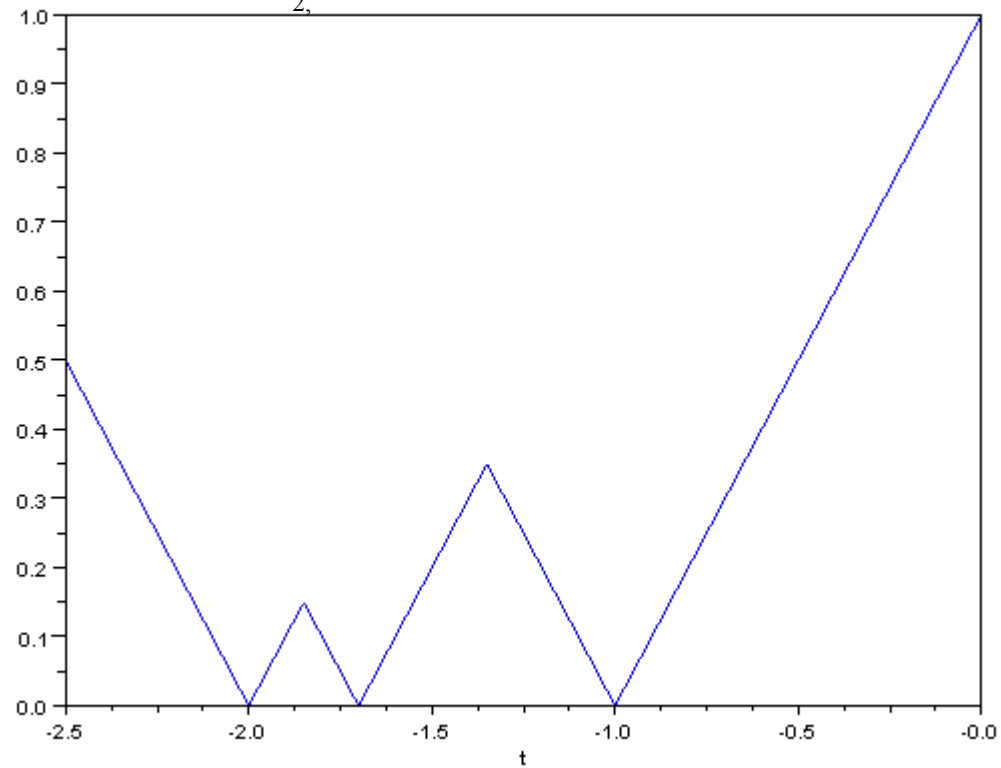
$$f_{a_i, B}(t) = d(a_i + t, B) = \min_{j \in J} d(a_i + t, b_j)$$

$$f_{b_j, A}(t) = d(b_j, A + t) = \min_{i \in I} d(a_i + t, b_j)$$

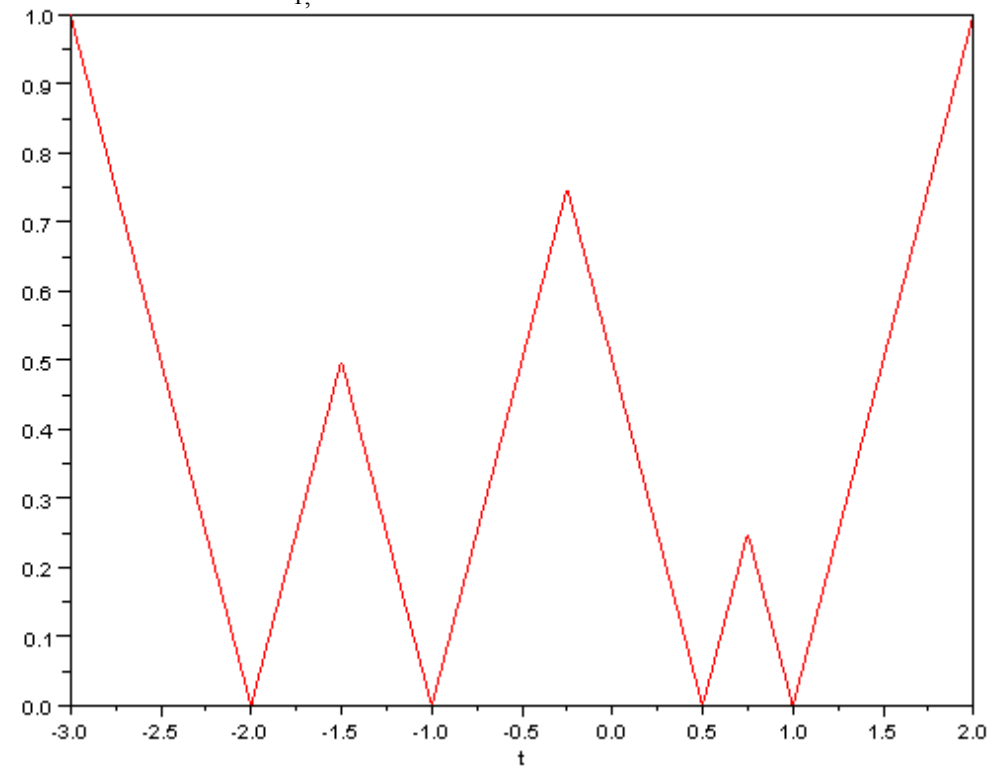
1. Compute the m functions $f_{a_i, B}(t)$, each having n minima.
2. Compute the n functions $f_{b_j, A}(t)$, each having m minima.
3. Compute the **upper envelope** of the $(m+n)$ functions.
 - Sort the spikes according to their left endpoints and add them from left to right.
 - We have **$O(mn)$** segments, need **$O(mn \cdot \log(mn))$** time to sort them.
4. Find the global minimum.

$$A=\{0, 0.5, 2, 3\} , B=\{0, 0.3, 1\}$$

$$a = 2 , f_{2,B}(t)$$

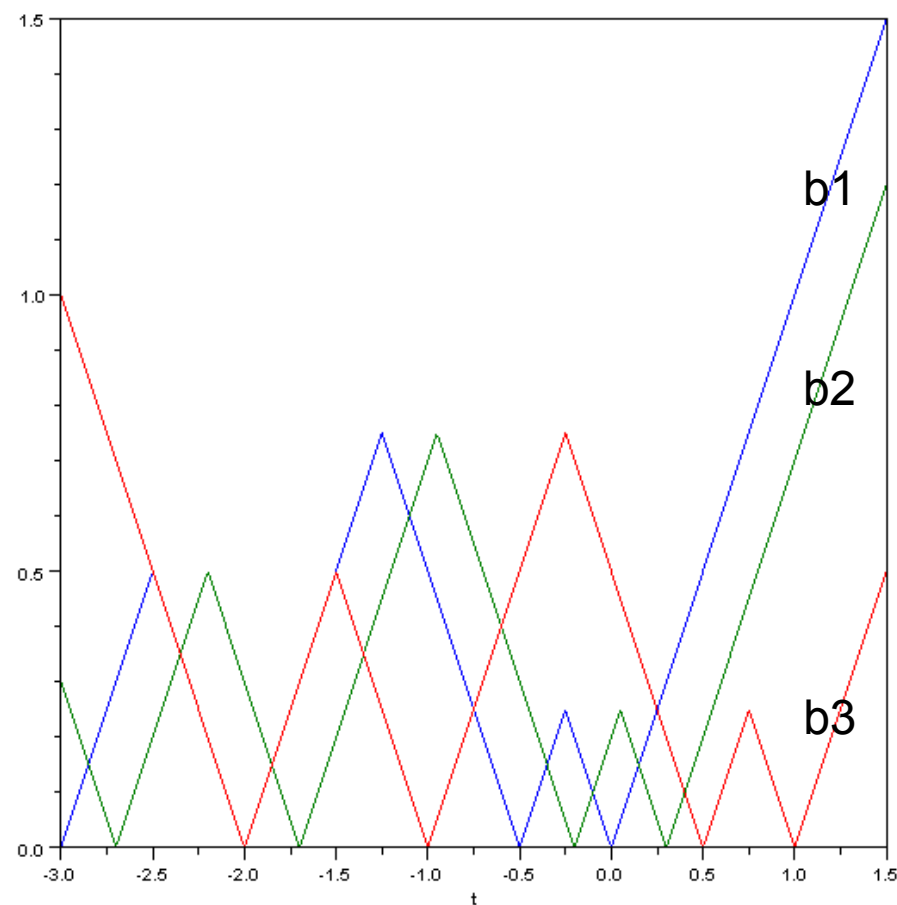
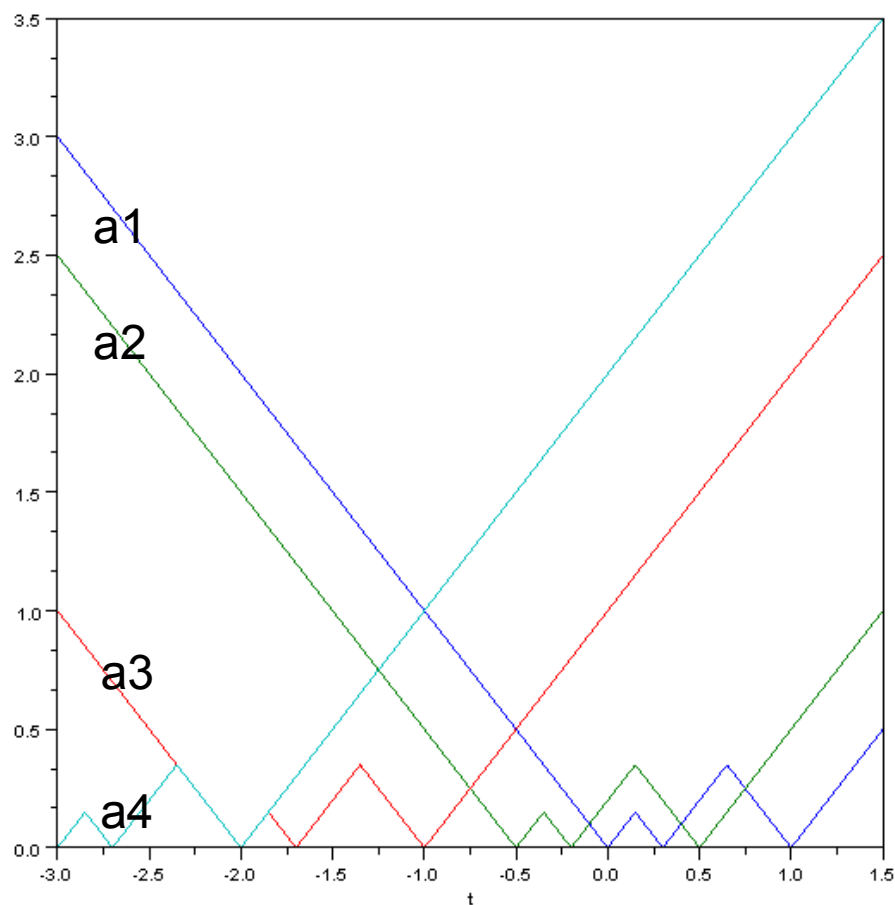


$$b = 1 , f_{1,A}(t)$$



The slope of all segments = 1 or -1

$$A=\{0, 0.5, 2, 3\} , B=\{0, 0.3, 1\}$$



When $t = -1$, the minimum Hausdorff distance = 1

Rote's Algorithm(Optimal)

Time Complexity: $O((m+n)*\log(m+n))$

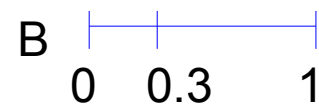
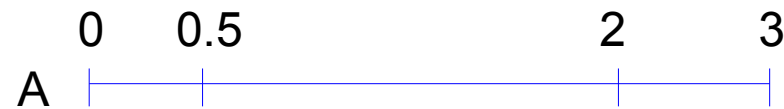
- 1.Sort A and B in increasing order, respectively.
- 2.Construct A^* and B^* from A and B.
- 3.Shift A^* and B^* such that the endpoints of them are superimposed.
- 4.Find spikes of each $f_{a_i, B^*}(t)$ above the function $g(t) = |t|$
- 5.Sort the spikes found by step 4.

Because each $(m+n)$ functions contributes at most two edges to $g(t)$, we have at most $O(m+n)$ spikes, need $O((m+n)*\log(m+n))$ time to sort them.

- 6.Compute the minimum Hausdorff distance under translation.

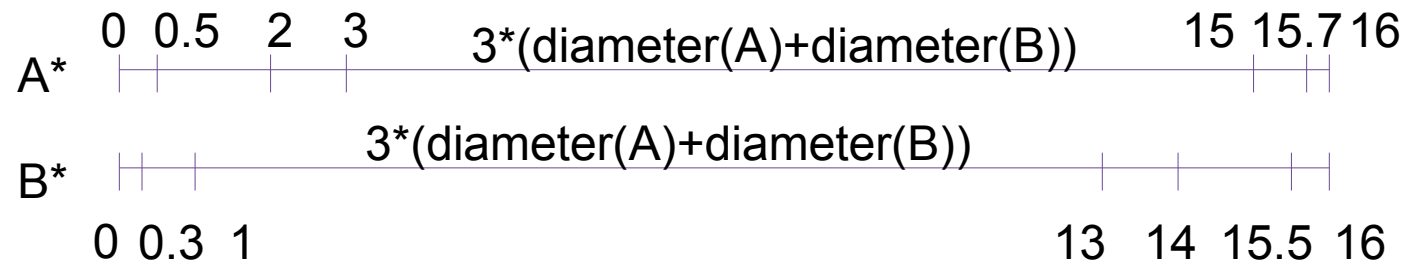
$$A = \{0, 0.5, 2, 3\},$$

$$B = \{0, 0.3, 1\}$$



$$A^* = \{0, 0.5, 2, 3, 15, 15.7, 16\}$$

$$B^* = \{0, 0.3, 1, 13, 14, 15.5, 16\}$$



Hausdorff distance $d(A, B) = \max_{a_i \in B^*} f_{a_i}(t)$, a_i belongs to A^*

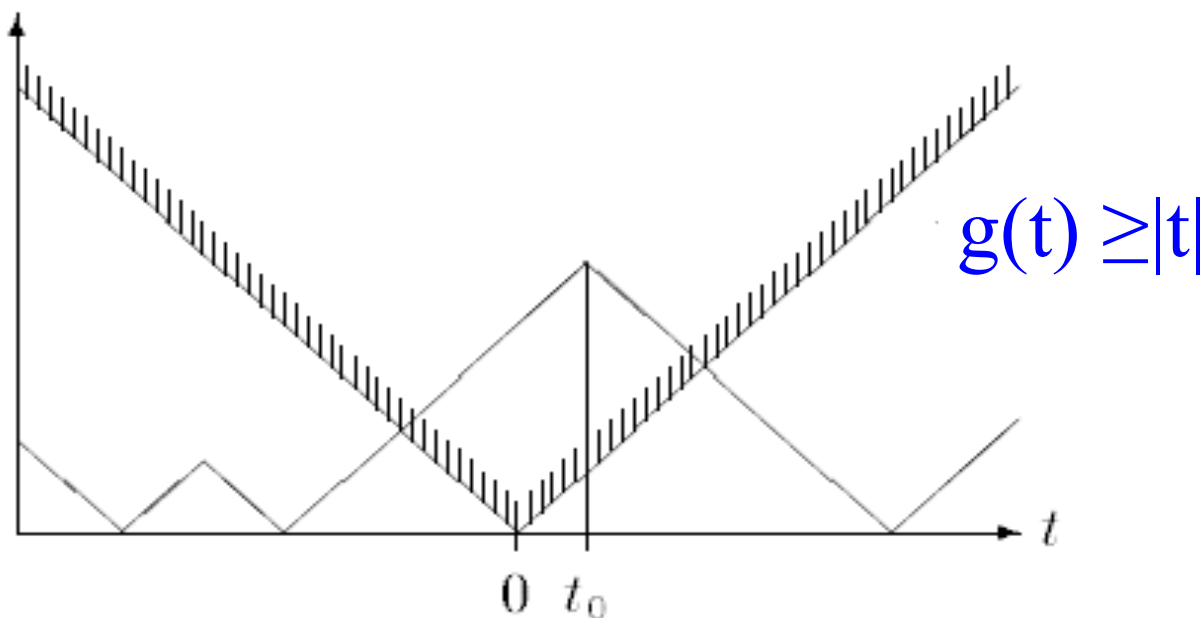
If we shift A^* by t units, the left or the right endpoint will contribute at least $|t|$ to the Hausdorff distance.

$$g(t) = \max_{a_i \in A^*} f_{a_i, B^*}(t), \quad a_i \text{ belongs to } A^*$$

We can get $g(t) \geq |t|$ (low bound)

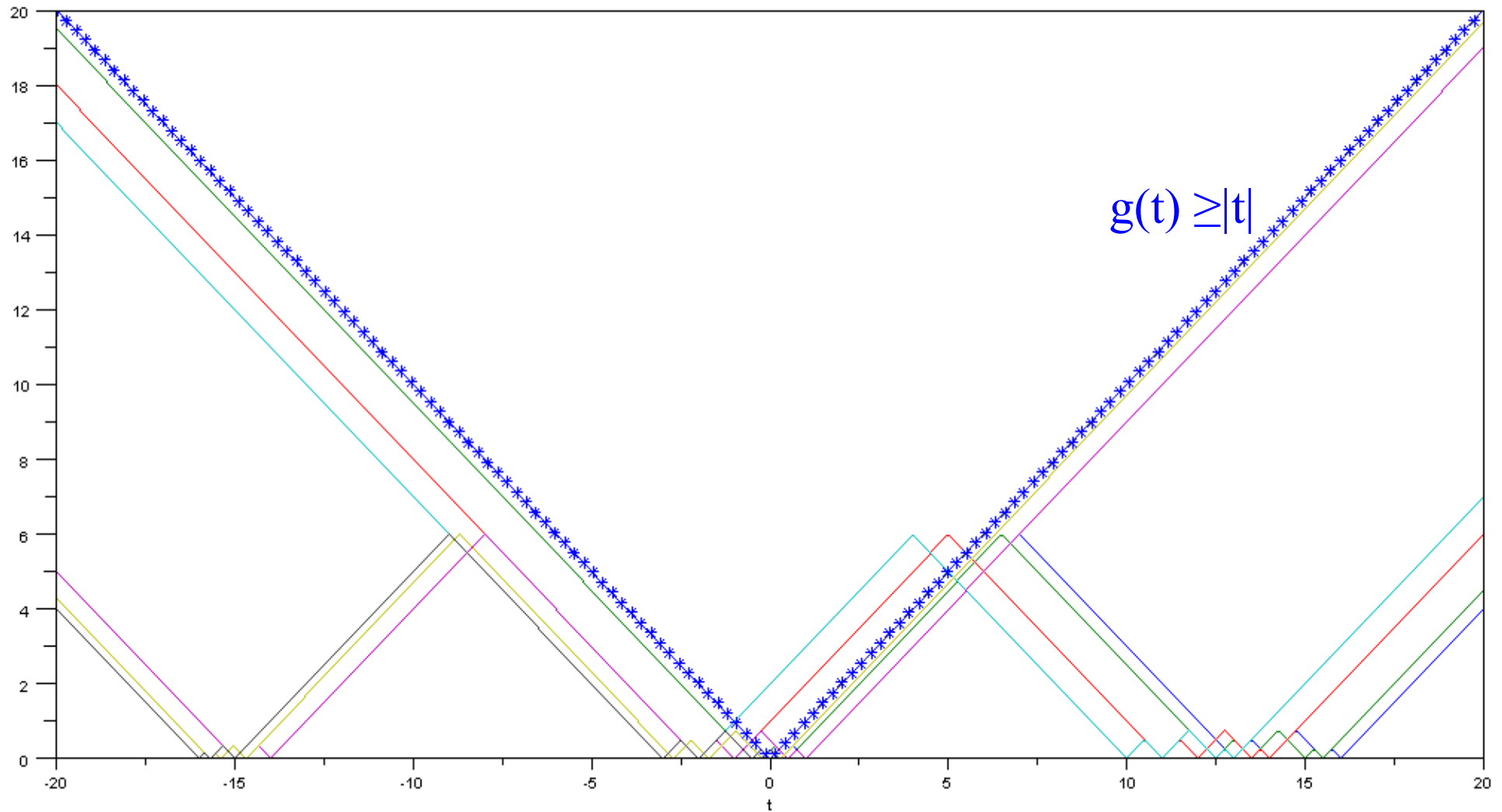
Lemma

Each of the $(m+n)$ functions $d(a_i + t, B^*)$ contributes **at most two edges** to the function $g(t)$



$$A^* = \{0, 0.5, 2, 3, 15, 15.7, 16\}$$

$$B^* = \{0, 0.3, 1, 13, 14, 15.5, 16\}$$



When $t = -1$, the minimum Hausdorff distance = 1

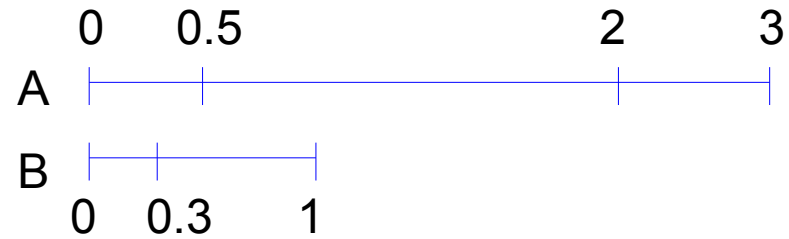
The new Algorithm(Optimal)

Time Complexity: $O((m+n)*\log(m+n))$

1. Denote A as the set with **larger diameter** and B as the other set.
2. Sort A and B in increasing order, respectively.
3. Shift A and B such that $a_m = b_1 = 0$.
4. Find spikes of $f_{a_i, B}(t)$ and $f_{b_j, A}(t)$ above the function **$f(t)$**
5. Sort the spikes found by step 4. **$O((m+n)*\log(m+n))$**
6. Compute the minimum Hausdorff distance under translation.

$$A = \{0, 0.5, 2, 3\}$$

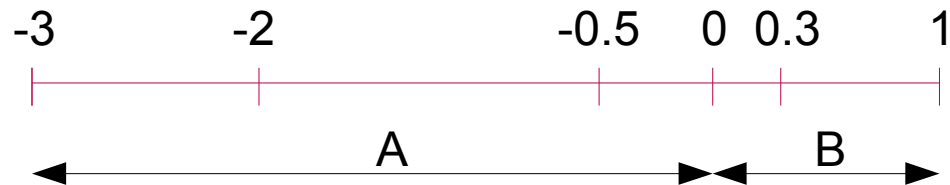
$$B = \{0, 0.3, 1\}$$



Shift A and B such that $a_m = b_1 = 0$

$$A = \{-3, -2, -0.5, 0\}$$

$$B = \{0, 0.3, 1\}$$



Theorem 3. If $|a_1 - a_m| \geq |b_1 - b_n|$, then

$$H(A + t, B) = \begin{cases} f_{a_1, B}(t), & \text{if } t < 0; \\ f_{a_m, B}(t), & \text{if } t > -a_1 + b_n. \end{cases}$$

If $|a_1 - a_m| < |b_1 - b_n|$, then

$$H(A + t, B) = \begin{cases} f_{b_n, A}(t), & \text{if } t < 0; \\ f_{b_1, A}(t), & \text{if } t > -a_1 + b_n. \end{cases}$$

When $t < 0$

$$\begin{aligned} f_{a_i, B}(t) &= \min d(a_i + t, B) = \min (t, -a_i + b_j), \text{ for all } j \text{ belongs to } J \\ &= \min((-a_i + b_j) - t) = (-a_i + b_1 - t) = -a_i - t \end{aligned}$$

because $a_1 < a_2 < \dots < a_m = 0$, $f_{a_1, B}(t) > f_{a_2, B}(t) > \dots > f_{a_m, B}(t)$

When $t > (b_n - a_1)$

$$\begin{aligned} f_{a_i, B}(t) &= \min d(a_i + t, B) = \min (t, -a_i + b_j), \text{ for all } j \text{ belongs to } J \\ &= \min((t - b_j) + a_i) = (t - b_n) + a_i > 0 \end{aligned}$$

Because $a_1 < a_2 < \dots < a_m = 0$, $f_{a_1, B}(t) < f_{a_2, B}(t) < \dots < f_{a_m, B}(t)$

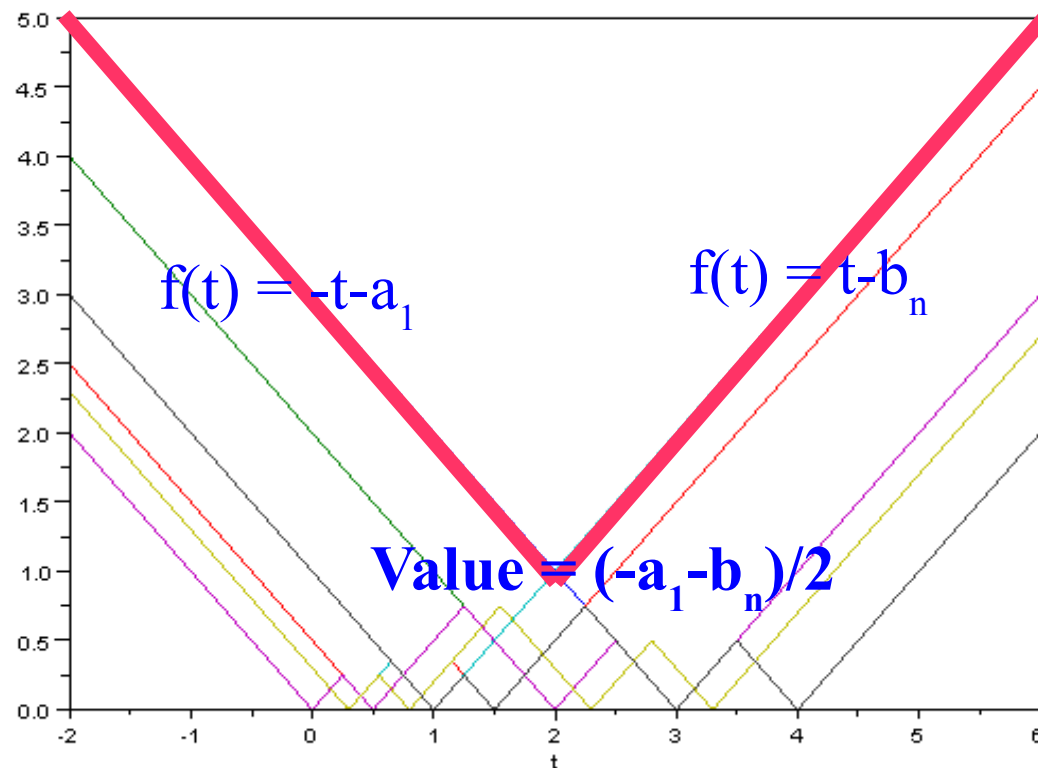
Assume $|a_1 - a_m| \geq |b_1 - b_n|$, let $f(t)$ be

$$f(t) = \begin{cases} f_{a_1, B}(t), & \text{if } t \leq \frac{b_n - a_1}{2}; \\ f_{a_m, B}(t), & \text{if } t > \frac{b_n - a_1}{2}. \end{cases}$$

Indeed $f(t) = \max \{f_{a_1, B}(t), f_{a_m, B}(t), f_{b_1, A}(t), f_{b_n, A}(t)\}$

$$A = \{-3, -2, -0.5, 0\}$$

$$B = \{0, 0.3, 1\}$$



How to Find spikes of $f_{a_i, B}(t)$ and $f_{b_j, A}(t)$ above the function $f(t)$

Lemma

$$\exists t, f_{a_i, B}(t) > f(t) \Leftrightarrow$$

$$\exists ! j \in J \ni (-a_i + b_j) > -a_1 \text{ and } (-a_i + b_{j-1}) < b_n$$

proof:

$$f_{a_i, B}(t) = d(t, -a_i + B) = \min_{j \in J} d(t, -a_i + b_j)$$

$$f_{a_i, B}(t) = 0 \Leftrightarrow t \in B - a_i$$

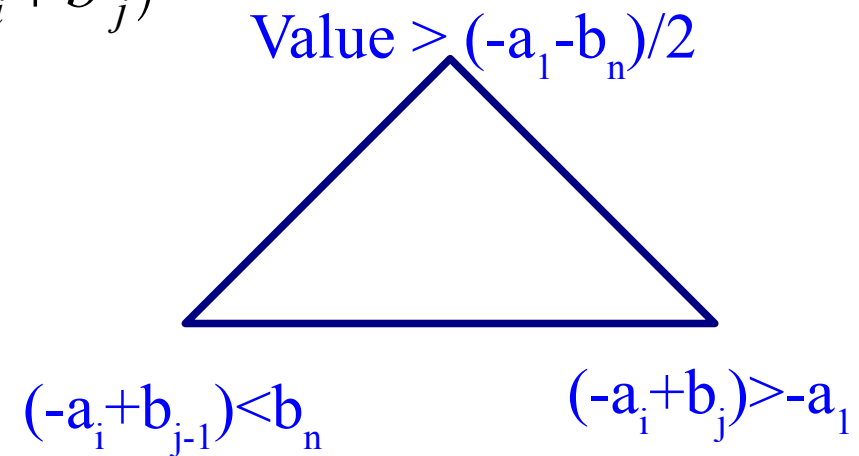
$$f_{a_i, B}(-a_i + b_j) = 0$$

$$f_{a_i, B}(-a_i + b_{j-1}) = 0$$

$$\text{When } (-a_i + b_{j-1}) \leq t \leq (-a_i + b_j)$$

$f_{a_i, B}(t)$ forms a spike.

The spike crosses $f(t) \Leftrightarrow (-a_i + b_j) > -a_1 \text{ and } (-a_i + b_{j-1}) < b_n$



How to Find spikes of $f_{a_i, B}(t)$ and $f_{b_j, A}(t)$ above the function $f(t)$

Lemma

$$\exists t, f_{b_j, A} > f(t) \Leftrightarrow$$

$$\exists ! i \in I \ni (b_j - a_i) > -a_1 \text{ and } (-b_j + a_{i+1}) < b_n$$

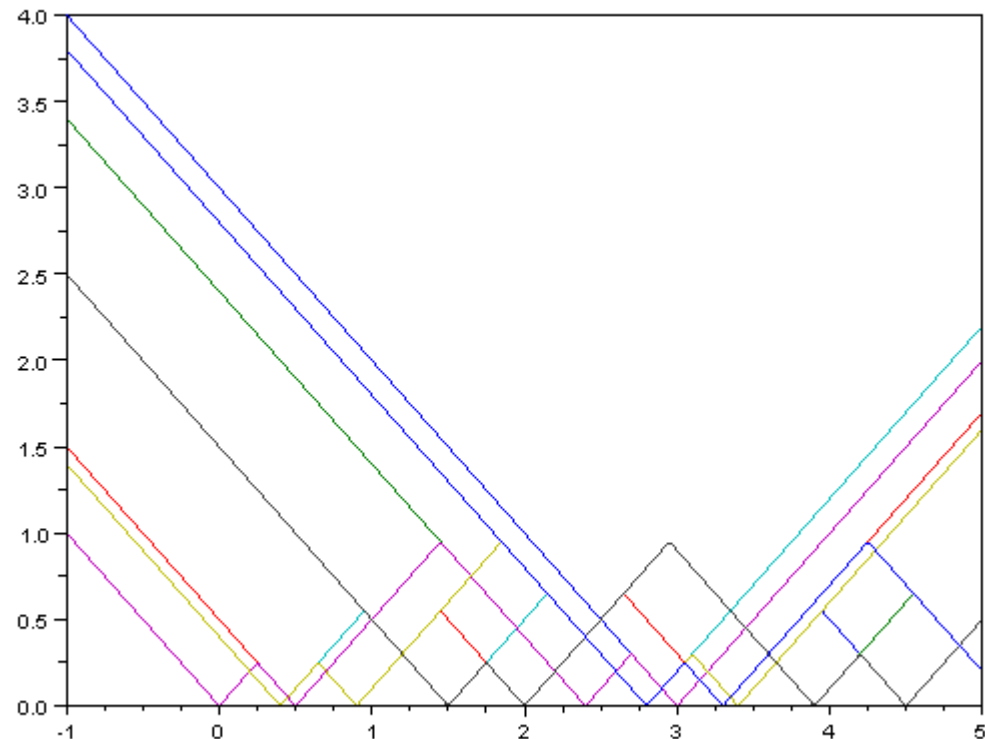
Let S contains the elements that satisfy previous and this lemma.

All the spikes possibly contribute to the $H(A+t, B)$

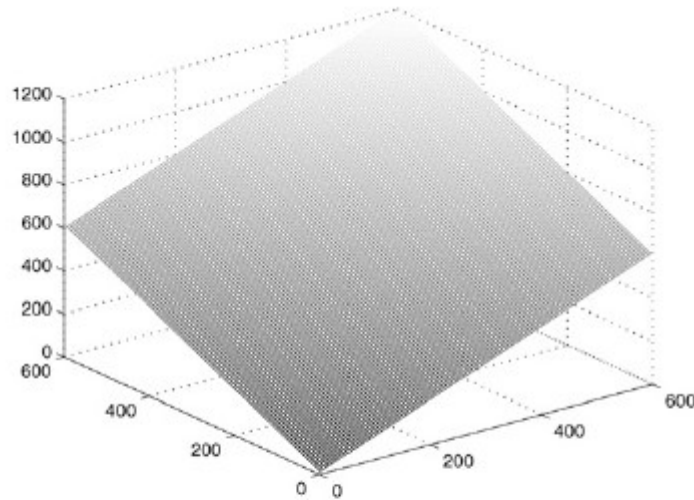
Another Example :

$$A = \{-3, -2.4, -0.5, 0\}$$

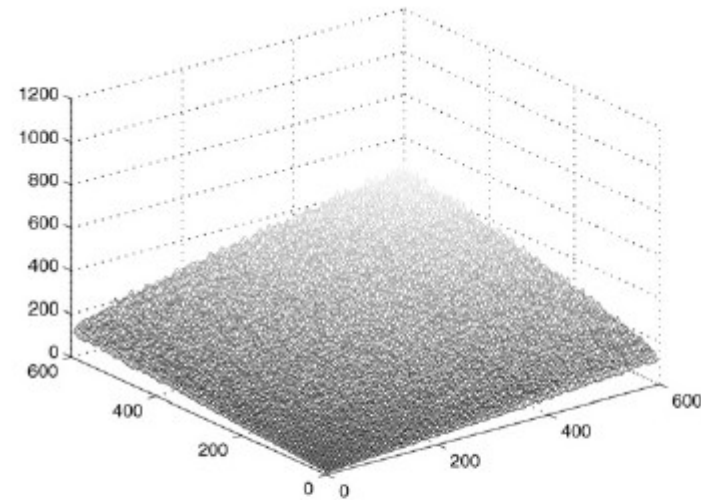
$$B = \{0, 0.3, 1, 2.8\}$$



Algorithm Comparison



(a)



(b)

Fig. 4. (a) shows the mean value of the number of spikes found by Rote's algorithm for each pair of sizes. (b) shows the mean value of the number of intervals found by our algorithm for each pair of sizes.

(a) uses Rote's algorithm, (b) uses the new algorithm

Sizes of set A and B from $\{5, 10 \dots 595, 600\}$

Randomly generated A and B in interval $[0,1]$ for 400 times.

Axis X, Y represent the sizes of set A and B

Axis Z represents the **number of spikes** found by algorithm

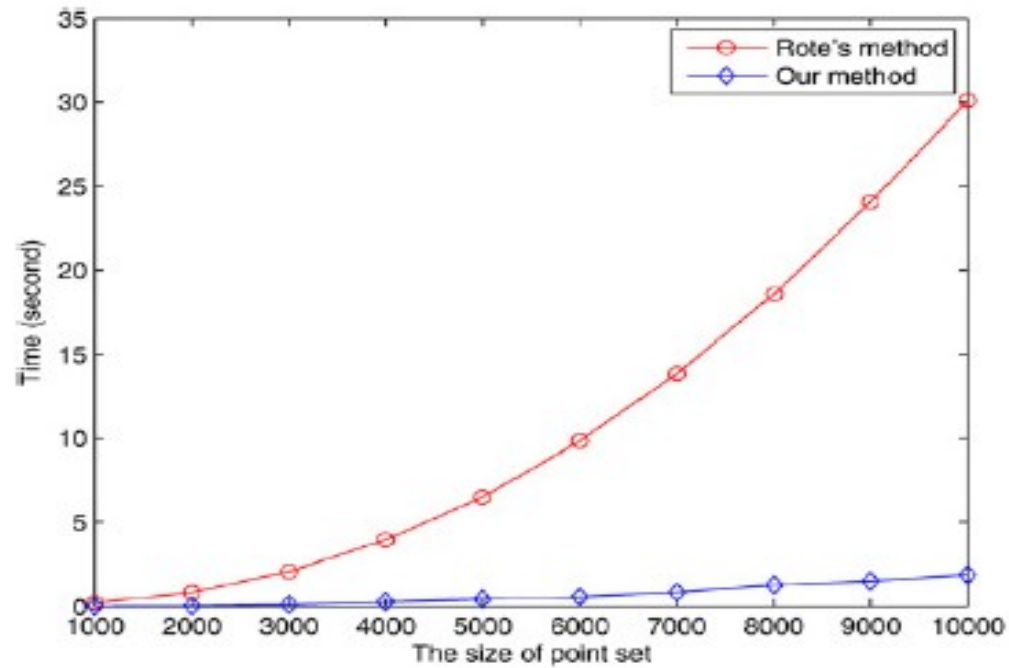


Table 2

This table shows the runtime of Rote's algorithm and our algorithm

		1000	2000	3000	4000	5000
Rote's algorithm	1000	0.2179	0.4630	0.8380	1.3668	2.0308
	2000	0.4676	0.8581	1.3946	2.0578	2.8702
	3000	0.8164	1.3998	2.0785	2.9052	3.9127
	4000	1.4017	2.0581	2.9073	3.9195	5.0566
	5000	2.0113	2.8898	3.9027	5.0410	6.4663
Our algorithm	1000	0.0149	0.0255	0.0433	0.0601	0.0930
	2000	0.0284	0.0480	0.0786	0.1149	0.1389
	3000	0.0468	0.0721	0.1307	0.1435	0.2552
	4000	0.0606	0.0955	0.1839	0.2468	0.3512
	5000	0.0904	0.1881	0.2573	0.2856	0.3702

The numbers in the first row and the second column are the sizes of the input sets. We computed the minimum Hausdorff distance of two random input sets with corresponding sizes for 200 times. Each entry in the table shows the average runtime (in seconds) spent by Rote's algorithm and ours.